The following steps and source codes elaborate how to manage API calls in a HyperApp application.

***Step 1: Define the initial state***
Start by defining the initial state of your application, including properties related to API data. For example:

```
const state = {
todos: [ ],
isLoading: false,
};
```

***Step 2: Define actions for API requests***
Create actions that handle API requests. Typically, these actions will trigger asynchronous operations and update the *state* accordingly. Here's an example of an action that fetches to-dos from an API:

```
const actions = {
fetchTodos: () => async (state,
actions) => {
actions.setLoading (true);
actions.setError(null);
try {
const response = await fetch('https://
api.example.com/todos');
const todos = await response.json();
actions.setTodos (todos);
} catch (error) {
actions.setError('Failed to fetch
todos');
console.error('Failed to fetch todos:',
error);
}
actions.setLoading (false);
},
setTodos: (todos) => ({ todos }),
setLoading: (isLoading) => ({ isLoading
}),
setError: (error) => ({ error }),
};
```

In the above example, *fetchTodos* is an asynchronous action that sets the *isLoading* state to true, performs the API request using *fetch*, and updates the *todos* state with the fetched data. If an error occurs during the request, it is logged to the console.

***Step 3: Define the view***
Update your *view* function to conditionally render components based on the *state*. For example, you can show a loading indicator when the API request is in progress. Here's an updated example:

```
const view = (state, actions) => (
<div>
{state.isLoading ? (
<div>Loading...</div>
) : (
<div>
<ul>
{state.todos.map((todo) => (
<li>{todo.title}</li>
))}
</ul>
<button onclick={actions.
fetchTodos}>Fetch Todos</button>
</div>
    )}
</div>
);
```

***Step 4: Mount the application***
Mount the application as before, using the updated *state*, *actions*, and *view* functions:

```
const main = app(state,
actions, view, document.
getElementById('app'));
```

Now, when the 'Fetch Todos' button is clicked, it will trigger the *fetchTodos* action, which will make the API request, update the state with the fetched to-dos, and re-render the view.

We can extend this example by adding more actions for different API requests, handling request parameters, error handling, and more. Additionally, you might want to consider using HyperApp effects, such as the http effect, or external libraries like Axios for more advanced API handling.

## Testing and debugging HyperApp applications

Testing and debugging are crucial steps in the development process of HyperApp applications to ensure that the application functions correctly and handles potential issues gracefully.

Here are some approaches and tools you can use for testing and debugging HyperApp applications.

***Unit testing:*** Write unit tests to verify the behaviour of individual components, actions, or utility functions in isolation. Tools like Jest or Mocha can be used for testing HyperApp applications. Unit tests help identify and fix bugs at an early stage and ensure that changes to the codebase don't introduce regressions.

***Integration testing:*** Perform integration tests to validate the interaction between different components, actions, and the overall application behaviour. Integration tests can be written using testing frameworks like Cypress or Selenium, which simulate user interactions and verify the application's functionality across multiple components.

***Debugging tools:*** HyperApp provides debugging tools that can help identify and fix issues. For example, you can use the Redux DevTools extension for debugging state changes, actions, and their payloads. HyperApp also has a built-in *app({ ... })* method for logging state changes and rendering performance optimisations.

***Console logging:*** Insert *console.log* statements at critical points in your code to log values, state changes, or function outputs during runtime. Console logging can help track the flow of execution and identify unexpected behaviour or incorrect data.

***Error handling:*** Implement proper error handling and error messages to make debugging easier. Use try/catch blocks around critical sections of code, especially asynchronous operations and API requests. Log or display meaningful error messages to assist in identifying and resolving issues.